

För att skapa en ny React-applikation med stöd för TypeScript och SASS och namnet "chatgpt", kan du följa dessa enkla steg:

[[TOC]]

Node

1. **Installera Node.js:** Se till att du har Node.js v.18 eller högre installerat på din dator. Om du inte har det, kan du ladda ner och installera det från [Node.js officiella webbplats](#).

Expo-cli

```
yarn add -g expo-cli
```

Skapa ett projekt med vite, react & typescript

Det kommer att fråga dig namnet på projektet, ramverket som ska användas och varianten (React och TypeScript).

```
yarn create vite
```

```
cd nameofproject
yarn
yarn dev
```

Sass

Den här kommandoraden `yarn add sass` används för att lägga till sass som ett beroende i ett projekt som använder Yarn för pakethantering.

```
yarn add sass
```

OpenAI

1. **Installera OpenAI-paketet:** För att lägga till OpenAI-stöd i din ChatGPT React-applikation behöver du installera OpenAI:s officiella npm-paket. Detta paket ger dig tillgång till OpenAI:s API, vilket gör det möjligt för din applikation att kommunicera med OpenAI:s tjänster, inklusive GPT-3 och andra AI-modeller. Här är stegen du behöver följa:

Kör följande kommando för att installera OpenAI-paketet via npm:

```
npm install openai
```

Detta kommando lägger till OpenAI-paketet till dina projektberoenden och laddar ner det till din `node_modules`-mapp.

2. **Kontrollera `package.json`:** Efter installationen, kontrollera din `package.json`-fil för att se till att OpenAI-paketet har lagts till i listan över beroenden.
3. **Använd OpenAI-paketet i din applikation:** Nu när OpenAI-paketet är installerat, kan du importera och använda det i din React-applikation för att kommunicera med OpenAI:s API.

Kom ihåg att du behöver en API-nyckel från OpenAI för att kunna göra API-anrop. Denna nyckel bör hanteras försiktigt och aldrig exponeras på klient-sidan av din applikation av säkerhetsskäl. `openAI key= sk-983skJfubNcSjlRNLIx9T3B1bkFJArLJyCXaWRKfZWdJCZJU`

Axios

För att installera Axios i din React ChatGPT-applikation och för att förstå vad Axios är, kan du följa dessa enkla steg:

Installation av Axios

1. **Öppna din terminal eller kommandotolk:** Starta terminalen eller kommandotolken på din dator.
2. **Navigera till din projektmapp:** Använd `cd` (change directory) för att navigera till mappen där ditt React-projekt är beläget. Till exempel:

```
cd path/to/your/chatgpt-project
```

3. **Installera Axios:** Kör följande kommando för att installera Axios i ditt projekt:

```
npm install axios
```

Detta lägger till Axios i ditt projekt och gör det möjligt för dig att använda det i din applikation.

Vad är Axios?

Axios är ett populärt JavaScript-bibliotek som används för att göra HTTP-förfrågningar från node.js eller XMLHttpRequests från webbläsaren. Det är speciellt användbart i React-applikationer för att kommunicera med externa API:er och webbtjänster. Här är några nyckelpunkter om Axios:

- **Enkel att använda:** Axios erbjuder en enkel och ren API för att skicka olika typer av HTTP-förfrågningar (som GET, POST, DELETE, PUT) och hantera svar.
- **Promisbaserat:** Axios använder JavaScript Promises, vilket gör det enkelt att hantera asynkrona operationer och fånga fel.

- **Stöd för moderna webbfunktioner:** Axios har stöd för moderna webbfunktioner som timeout, avbrytning av förfrågningar och automatisk omvandling av JSON-data.
- **Interceptors:** Axios ger möjlighet att använda interceptors som låter dig köra kod eller ändra förfrågningar och svar innan de hanteras av 'then' eller 'catch'.

Genom att använda Axios i din React ChatGPT-applikation kan du enkelt integrera med externa API:er som OpenAI för att hämta och skicka data. Det är ett kraftfullt verktyg som gör din webbutveckling mer effektiv och organiserad.

Källkodshantering

Att lägga till din applikation till ett befintligt Git-repository i Azure DevOps är en bra idé för versionshantering och samarbete. Här är en steg-för-steg-instruktion för hur du gör detta:

1. **Öppna din terminal eller kommandotolk:** Starta terminalen eller kommandotolken på din dator.
2. **Navigera till din projektmappe:** Använd `cd` (change directory) för att navigera till mappen där ditt React-projekt är beläget. Till exempel:

```
cd path/to/your/chatgpt-project
```

3. **Initiera ett Git-repository (om det inte redan är gjort):** Om du inte redan har ett git-repository initierat i din projektmappe, kör:

```
git init
```

4. **Lägg till filer till ditt lokala repository:** Innan du kopplar ditt lokala repository till Azure DevOps, se till att lägga till alla relevanta filer och göra en initial commit. Använd följande kommandon:

```
git add .  
git commit -m "Initial commit"
```

5. **Lägg till fjärr-repository (remote):** Använd följande kommando för att lägga till ditt Azure DevOps repository som en remote till ditt lokala Git-repository. Se till att ersätta URL:en med din specifika repository-URL:

```
git remote add origin url_till_ditt_git-repo_(url_hämtas_via_DevOps  
projektet/repositoriet)
```

Kom ihåg att hantera dina känsliga filer (som .env eller konfigurationsfiler) noggrant och se till att de inte pushas till ditt publika repository om de innehåller känslig information.

Git-flow

Git-flow är ett arbetsflöde som använder Git, ett versionshanteringssystem, för att organisera och hantera kodutvecklingsprocessen på ett strukturerat sätt. Tänk dig Git-flow som en uppsättning regler och rekommendationer för hur man använder Git på bästa sätt när man jobbar i ett projekt, speciellt i team. Här är de grundläggande koncepten:

1. **Huvudgrenar:** Det finns två permanenta grenar (branches) i Git-flow:

- **Master:** Denna gren innehåller kod som alltid är i skick att släppas (release-ready), alltså helt testad och stabil.
- **Develop:** Denna är huvudgrenen för utveckling där alla nya funktioner samlas innan de är redo att släppas.

2. **Stödgrenar:** För att stödja olika utvecklingsaktiviteter används ytterligare grenar:

- **Featuregrenar:** Skapas från 'develop' för att utveckla nya funktioner. När funktionen är klar, sammanfogas (mergas) den tillbaka med 'develop'.
- **Releasegrenar:** När 'develop' har nått en punkt där funktionerna ska släppas, skapas en releasegren. Eventuella buggar fixas här innan den sammanfogas med 'master' och 'develop'.
- **Hotfixgrenar:** Om en kritisk bugg upptäcks i 'master', skapas en hotfix-gren för att snabbt åtgärda problemet. Efter fixen sammanfogas den med både 'master' och 'develop'.

3. **Fördelar:**

- **Struktur:** Git-flow ger en tydlig struktur och rutiner som hjälper till att hålla ordning i projektet, speciellt när flera personer arbetar tillsammans.
- **Effektivitet:** Det underlättar parallell utveckling av olika funktioner och effektiv hantering av släpp (releases).

Bransch

Att skapa en lokal gren (branch) för att hantera dina ändringar är en grundläggande och viktig del av Git-arbetsflödet. Detta gör att du kan utveckla funktioner, fixa buggar eller göra andra ändringar i en isolerad miljö utan att påverka den huvudsakliga eller master-/main-grenen. Här är en detaljerad guide för att skapa en lokal gren, pusha ändringar till Azure DevOps och sedan skapa en pull request för att sammanfoga (merge) dessa ändringar med master-/main-grenen:

1. **Skapa en ny lokal gren:** Först, navigera till din projektmapp i terminalen. Skapa sedan en ny gren för dina ändringar. Byt ut `feature_branch` med ett passande namn för din gren som beskriver vad du arbetar med:

```
git checkout -b feature_branch
```

Detta skapar och växlar till en ny gren.

2. **Gör dina ändringar:** Gör de ändringar du behöver i denna gren. Detta kan innefatta att lägga till nya filer, redigera befintliga filer, fixa buggar, etc.

3. **Lägg till och commita dina ändringar:** När du är klar med dina ändringar, använd `git add` för att lägga till ändringarna till staging-området, och sedan `git commit` för att commita dem. Till exempel:

```
git add .  
git commit -m "Beskrivning av dina ändringar"
```

4. **Pusha din gren till Azure DevOps:** Pusha din lokala gren till Azure DevOps-repositoryt:

```
git push -u origin feature_branch
```

5. **Skapa en pull request:** Gå till ditt Git-repository i Azure DevOps. Du bör se en prompt eller en knapp för att "Skapa en pull request" för din nyligen pushade gren. Klicka på denna för att påbörja processen.
6. **Fyll i detaljerna för din pull request:** Ange en titel och en beskrivning för din pull request. Det är bra att vara tydlig och informativ här så att dina kollegor förstår vad ändringarna innebär och varför de är nödvändiga.
7. **Välj granskare:** Beroende på ditt team och arbetsflöde, kanske du vill tilldela en eller flera personer att granska din kod.
8. **Skapa pull requesten:** När allt är inställt, klicka på "Skapa pull request" (eller motsvarande knapp).
9. **Granskning och diskussion:** Dina kollegor (och du själv) kan nu granska och diskutera ändringarna i pull requesten. Detta kan innefatta kodgranskning, feedback, och eventuella justeringar baserat på teamets feedback.
10. **Sammanfoga pull requesten:** När din pull request har godkänts, kan du eller en teammedlem sammanfoga den med master-/main-grenen. Detta görs vanligtvis genom Azure DevOps-gränssnittet.
11. **Rensa upp efteråt:** Efter en lyckad sammanfogning, glöm inte att radera din feature-gren från både din lokala arbetsyta och fjärr-repositoryt om det inte längre behövs.

Kom ihåg att god kommunikation och tydliga commit-meddelanden är nyckeln till ett framgångsrikt Git-arbetsflöde.